Game.com - Sacred Tech Scroll

Overview
Memory Map
System Reset
About
Change Colors

System Components
CPU - SM8521 CPU Core

Game.com is essentially a Sharp SM8521 single-chip microcomputer with a display, speaker and user inputs. This document usually refers to the SM8521 specifically, but pertains to the Game.com system in the general sense.

Memory Map

There SM8521 memory bus is 16 bits in size and is organized by memory type:

`0x0000` - `0x007F`	Register file
`0x0080` - `0x03FF`	RAM
`0x0400` - `0x0FFF`	Unmapped
`0x1000` - `0x9FFF`	ROM
`0xA000` - `0xDFFF`	VRAM
`0xE000` - `0xFFFF`	Extended RAM

RAM is built-in, general-purpose working memory. It can be used for all read and write operations by the CPU.

VRAM is built-in memory used to display images. It is write-only from the perspective of the CPU.

Editor's Note: Research is needed to determine what gets read by the CPU from VRAM.

Editor's Note: Research is needed to determine how the extended RAM region is used, if at all.

Register File

The *register file* contains system control registers, I/O ports and the CPU registers (which are memory-mapped). It has the following contents:

0x0000	`r0`, `rr0` (high 8 bits)	General-purpose registers [note]
0x0001	`r1`, `rr0` (low 8 bits)	General-purpose registers
0x0002	`r2`, `rr2` (high 8 bits)	General-purpose registers
0x0003	`r3`, `rr2` (low 8 bits)	General-purpose registers
0x0004	`r4`, `rr4` (high 8 bits)	General-purpose registers

0x0005	`r5`, `rr4` (low 8 bits)	General-purpose registers
0x0006	`r6`, `rr6` (high 8 bits)	General-purpose registers
0x0007	`r7`, `rr6` (low 8 bits)	General-purpose registers
0x0008	`r8`, `rr8` (high 8 bits)	General-purpose registers
0x0009	`r9`, `rr8` (low 8 bits)	General-purpose registers
0x000A	`r10`, `rr10` (high 8 bits)	General-purpose registers
0x000B	`r11`, `rr10` (low 8 bits)	General-purpose registers
0x000C	`r12`, `rr12` (high 8 bits)	General-purpose registers
0x000D	`r13`, `rr12` (low 8 bits)	General-purpose registers
0x000E	`r14`, `rr14` (high 8 bits)	General-purpose registers
0x000F	`r15`, `rr14` (low 8 bits)	General-purpose registers
0x0010	IE0	Interrupt Enable Register 0
0x0011	IE1	Interrupt Enable Register 1
0x0012	IR0	Interrupt Request Register 0
0x0013	IR1	Interrupt Request Register 1
0x0014	P0	PIO Data Register 0
0x0015	P1	PIO Data Register 1
0x0016	P2	PIO Data Register 2
0x0017	P3	PIO Data Register 3
0x0018	Unmapped	
0x0019	SYS	System Configuration Register
0x001A	CKC	Clock Change Register
0x001B	Unmapped	
0x001C	`SPH`, `SP` (high 8 bits)	Stack pointer
0x001D	`SPL`, `SP` (low 8 bits)	Stack pointer
0x001E	PS0	Processor Status Register 0
0x001F	PS1	Processor Status Register 1
0x0020	P0C	PIO Control Reigser 0
0x0021	P1C	PIO Control Reigser 1
0x0022	P2C	PIO Control Reigser 2
0x0023	P3C	PIO Control Reigser 3
0x0024	MMU0	MMU Data Register 0
0x0025	MMU1	MMU Data Register 1
0x0026	MMU2	MMU Data Register 2
0x0027	MMU3	MMU Data Register 3
0x0028	MMU4	MMU Data Register 4
0x0029	Unmapped	
0x002A	Unmapped	
0x002B	URTT	UART Transmit Data Register
0x002C	URTR	UART Receive Data Register
0x002D	URTS	UART Status Data Register
0x002E	URTC	UART Control Data Register
0x002F	Unmapped	

0x0030	LCC	LCD Control/Status Register
0x0031	LCH	Display Horizontal Timing Register
0x0032	LCV	Display Vertical Timing Register
0x0033	Unmapped	
0x0034	DMC	DMA Control Register
0x0035	DMX1	DMA Source X Coordinate Register
0x0036	DMY1	DMA Source Y Coordinate Register
0x0037	DMDX	DMA Source X Width Register
0x0038	DMDY	DMA Source Y Width Register
0x0039	DMX2	DMA Destination X Coordinate Register
0x003A	DMY2	DMA Destination Y Coordinate Register
0x003B	DMPL	DMA Palette Register
0x003C	DMBR	DMA ROM Bank Register
0x003D	DMVP	DMA VRAM Page Register
0x003E	Unmapped	
0x003F	Unmapped	
0x0040	SGC	SG Control Register
0x0041	Unmapped	
0x0042	SG0L	SG0 Output Level Control Register
0x0043	Unmapped	
0x0044	SG1L	SG1 Output Level Control Register
0x0045	Unmapped	
0x0046	SG0TH	SG0 Time Constant Register (high 8 bits)
0x0047	SG0TL	SG0 Time Constant Register (low 8 bits)
0x0048	SG1TH	SG1 Time Constant Register (high 8 bits)
0x0049	SG1TL	SG1 Time Constant Register (low 8 bits)
0x004A	SG2L	SG2 Output Level Control Register
0x004B	Unmapped	
0x004C	SG2TH	SG2 Time Constant Register (high 8 bits)
0x004D	SG2TL	SG2 Time Constant Register (low 8 bits)
0x004E	SGDA	SG D/A Direct Output Register
0x004F	Unmapped	
0x0050	TM0C	Timer Control Register 0
0x0051	TM0D	Timer Data Register 0
0x0052	TM1C	Timer Control Register 1
0x0053	TM1D	Timer Data Register 1
0x0054	CLKT	Clock Timer
0x0055	Unmapped	
0x0056	Unmapped	
0x0057	Unmapped	
0x0058	Unmapped	
0x0059	Unmapped	
0x005A	Unmapped	

0x005B	Unmapped	
0x005C	Unmapped	
0x005D	Unmapped	
0x005E	WDT	Watchdog Timer Register
0x005F	WDTC	Watchdog Timer Control Register
0x0060	SG0W0	SG0 Waveform Reigster 0
0x0061	SG0W1	SG0 Waveform Reigster 1
0x0062	SG0W2	SG0 Waveform Reigster 2
0x0063	SG0W3	SG0 Waveform Reigster 3
0x0064	SG0W4	SG0 Waveform Reigster 4
0x0065	SG0W5	SG0 Waveform Reigster 5
0x0066	SG0W6	SG0 Waveform Reigster 6
0x0067	SG0W7	SG0 Waveform Reigster 7
0x0068	SG0W8	SG0 Waveform Reigster 8
0x0069	SG0W9	SG0 Waveform Reigster 9
0x006A	SG0W10	SG0 Waveform Reigster 10
0x006B	SG0W11	SG0 Waveform Reigster 11
0x006C	SG0W12	SG0 Waveform Reigster 12
0x006D	SG0W13	SG0 Waveform Reigster 13
0x006E	SG0W14	SG0 Waveform Reigster 14
0x006F	SG0W15	SG0 Waveform Reigster 15
0x0070	SG1W0	SG1 Waveform Reigster 0
0x0071	SG1W1	SG1 Waveform Reigster 1
0x0072	SG1W2	SG1 Waveform Reigster 2
0x0073	SG1W3	SG1 Waveform Reigster 3
0x0074	SG1W4	SG1 Waveform Reigster 4
0x0075	SG1W5	SG1 Waveform Reigster 5
0x0076	SG1W6	SG1 Waveform Reigster 6
0x0077	SG1W7	SG1 Waveform Reigster 7
0x0078	SG1W8	SG1 Waveform Reigster 8
0x0079	SG1W9	SG1 Waveform Reigster 9
0x007A	SG1W10	SG1 Waveform Reigster 10
0x007B	SG1W11	SG1 Waveform Reigster 11
0x007C	SG1W12	SG1 Waveform Reigster 12
0x007D	SG1W13	SG1 Waveform Reigster 13
0x007E	SG1W14	SG1 Waveform Reigster 14
0x007F	SG1W15	SG1 Waveform Reigster 15

[note] The general-purpose registers can be relocated by configuring the `RP` field in PS0, allowing them to begin up to address `0x00F8`.

ROM

Read-only memory (ROM) is non-volatile memory that serves as the source for data assets and

program code. ROM is partitioned into 8 KiB banks, up to 5 of which are mapped into the memory bus at any given time in the ROM address range `0x1000` - `0x9FFF`.

ROM is managed through the following registers:

0x0024 MMU0 MMU Data Register 0
0x0025 MMU1 MMU Data Register 1
0x0026 MMU2 MMU Data Register 2
0x0027 MMU3 MMU Data Register 3
0x0028 MMU4 MMU Data Register 4

All five registers are read/write and store an unsigned 8-bit bank number. The bank number specifies the index of the 8 KiB portion of the ROM data, where bank 0 is the first 8 KiB of the data and so-on. The maximum size of ROM when containing all 256 banks is 2 MiB.

ROM banks 0 through 31 correspond to a 256 KiB internal system ROM and banks 32 through 255 correspond to cartridge ROM. Bank 32 still begins after the first 256 KiB of cartridge ROM, rendering that data inaccessible. Cartridges with less than 2 MiB of ROM may still be able to represent their full contents through address mirroring.

Each register controls a different subset of the ROM address range:

MMU0 `0x0000` - `0x1FFF` [note]
MMU1 `0x2000` - `0x3FFF`
MMU2 `0x4000` - `0x5FFF`
MMU3 `0x6000` - `0x7FFF`
MMU4 `0x8000` - `0x9FFF`

[note] Although addresses `0x0000` - `0x0FFF` do not correspond to ROM from the perspective of the CPU, it is still possible to access ROM in this range via DMA.

At system reset, addresses `0x1000` - `0x1FFF` are mapped to an internal 4 KiB ROM that contains a boot program. The first time `MMU0` is written, regular ROM mapping takes over and the boot ROM can no longer be accessed until the next reset.

Editor's Note: Research is needed to determine the ROM mapping at `0x0000` - `0x0FFF` while the boot ROM is active. According to the initial value of `MMU0`, it should be the lowest 4 KiB of system ROM bank 0.

CPU - SM8521 CPU Core

Overview
Specifications
Register Set
Operands

Instruction Set
Memory and Register

Arithmetic
Bitwise
Jump and Control

Appendix
Opcode Map

Specifications

The CPU core of the SM8521 is called SM85CPU:

Name:	SM85CPU
Bus width:	16 bits
Clock speed:	Unknown
Instruction cache:	None
Instruction set:	CISC
Manufacturer:	Sharp Corporation
Word size:	8 bits with some 16-bit operations
Year:	1995

Although it shares its name with the CPU cores of other SM8500 series computers, the SM8521 CPU core has unique features.

Register Set

All CPU registers except PC are mapped to a region of memory called the *register file*. Register operations will act on the corresponding values in memory, and memory operations can affect the contents of CPU registers.

The following registers are present:

`r0 - r15`	8 bits	General-purpose
`rr0 - rr14`	16 bits	General-purpose (pair)
PC	16 bits	Program counter
PS0	8 bits	Processor status 0
PS1	8 bits	Processor status 1
SP	16 bits	Stack pointer
SYS	8 bits	System configuration register

General-Purpose

There are 16 bytes designated as general-purpose registers, on which most CPU instructions operate. They can be used in any context—be it 8- or 16-bit—such as in arithmetic operations, as memory addresses, etc.

When being used as 8-bit registers, the general-purpose registers are named `r0` through `r15`. When being used as 16-bit *register pairs*, they are named `rr0` through `rr14` (even numbers only) and combine both the numbered byte and the following byte.

The `RP` field in PS0 can be used to relocate the 16 bytes of general-purpose registers to places other than their default position in the register file.

`PC` - Program Counter

Holds the address of the instruction about to be executed. `PC` is automatically incremented as instruction bytes are fetched from memory.

`PS0` - Processor Status 0

Configures high-level settings for the processing of general-purpose registers and masking of interrupts. Is a byte with the following format:

7	3	2	0
RP		IM	
5		3	

`RP` is the *register pointer* and specifies the base address of the general-purpose registers in memory. Shift the value of `RP` left by 3 bits to yield the bus address of the general-purpose registers, where the default is 0 and the high-order bits are all clear.

`IM` is the *interrupt mask* and specifies which interrupt levels are masked (ignored). Is one of the following values:

0 No interrupts are masked.
1 No interrupts are masked.
2 Interrupt level 8 is masked.
3 Interrupt levels 7 and 8 are masked.
4 Interrupt levels 6 through 8 are masked.
5 Interrupt levels 5 through 8 are masked.
6 Interrupt levels 4 through 8 are masked.
7 Interrupt levels 3 through 8 are masked.

`PS1` - Processor Status 1

Contains conventional status flags. Is a byte with the following format:

7	6	5	4	3	2	1	0
C	Z	S	V	D	H	B	I
1	1	1	1	1	1	1	1

C	Carry	Set when an operation produces a carry or borrows.
Z	Zero	Set when the result of an operation is zero.

S	Sign	Set when the result of an operation is negative.
V	Overflow	Set when an operation results in integer overflow.
D	Decimal adjustment	Set after a subtraction.
H	Half carry	Similar to `C`, but applies with respect to the lowest 4 bits of the operation.
B	Bit	Serves as an operand in 1-bit operations.
I	Interrupt enable	When set, interrupts will be processed.

During the operation of the branch and call instructions, certain flag state combinations are checked in `PS1`.

`SP` - Stack Pointer

Points to the most recent data added to the stack.

The `SPC` field in SYS can be used to configure whether `SP` functions as an 8-bit or 16-bit register. When acting as an 8-bit register, the highest 8 bits of stack addresses are treated as clear.

The individual bytes of `SP` may be referenced as `SPH` for the high 8 bits or `SPL` for the low 8 bits.

Editor's Note: Research is needed to confirm that the high bits of stack addresses are clear when `SPC` is clear.

`SYS` - System Configuration Register

Configures auxiliary system features. Is a byte with the following format:

```
7 6   5 3 2 1 0
- SPC -   MCNF
1 1   3   3
```

-		These bits have no significance and should be cleared.
SPC	Stack pointer configuration	When clear, only the lower 8 bits of SP are used. When set, all 16 bits are used.
MCNF	Memory configuration	Configures the expansion memory region (see below).

When `SPC` is clear, the upper 8 bits of stack addresses are treated as clear.

`MCNF` controls the expansion memory at addresses `0x1000` - `0xFFFF`. Must be one of the following values:

0 Disable expansion memory.
6 Enable expansion memory.

Editor's Note: Research is needed to determine whether the `-` bits actually do anything.

Editor's Note: Research is needed to confirm that the high bits of stack addresses are clear when `SPC` is clear.

Editor's Note: Research is needed to determine what happens when accessing expansion memory while `MCNF=0`.

Editor's Note: Research is needed to determine what happens for other values of `MCNF`.

Operands

An instruction's operands specify the target(s) of the operation. When multiple operands are present, the operation's destination is listed on the left and the source(s) on the right. Depending on the instruction, data for different kinds of operands may be present, encoded directly into the instruction's machine code.

Editor's Note: Research is needed to determine whether `RP` affects only 3-bit indexed operands or all register file accesses in the `0x00` - `0x0F` range.

Operands

3-Bit Register Indexes
Condition
Register
Register Pair
Register File
Register Pair File
Register Indirect
Register Pair Indirect
Register Indirect Auto Increment
Register Pair Indirect Auto Increment
Register Indirect Auto Decrement
Register Pair Indirect Auto Decrement
Register Index
Register Pair Index
Index Indirect
Immediate
Immediate Long
Bit
Relative
Direct
Direct Short
Direct Special Page
Direct Indirect

3-Bit Register Indexes

Many instructions specify general-purpose registers or register pairs using 3-bit fields to reduce the required space used. The values of these 3-bit fields correspond to the following registers and register pairs:

Value	8-bit	16-bit
0	r0	rr0
1	r1	rr8
2	r2	rr2
3	r3	rr10

4	r4	rr4
5	r5	rr12
6	r6	rr6
7	r7	rr14

Condition

The BR and JMP instructions can conditionally move the program flow to another address depending on certain combinations of flags in PS1. Condition codes are 4 bits:

Value	Mnemonic	Condition	Test
0	F	Never	if false
1	LT	Signed less than	if (v XOR s) = 1
2	LE	Signed less than or equal	if ((v XOR s) OR z) = 1
3	ULE	Unsigned less than or equal	if (c OR z) = 1
4	OV	Overflow	if v = 1
5	MI	Minus	if s = 1
6	EQ, Z	Equal, Zero	if z = 1
7	ULT, C	Unsigned less than, Carry	if c = 1
8	T	Always	if true
9	GE	Signed greater than or equal	if (v XOR s) = 0
10	GT	Signed greater than	if ((v XOR s) OR z) = 0
11	UGT	Unsigned greater than	if (c OR z) = 0
12	NOV	Not overflow	if v = 0
13	PL	Plus	if s = 0
14	NE, NZ	Not equal, Not zero	if z = 0
15	UGE, NC	Unsigned greater than or equal, Not carry	if c = 0

Certain conditions have more than one mnemonic. In these situations, either mnemonic can be used and the two are interchangeable.

Register
Register Pair

Specifies a general-purpose register or register pair using a 3-bit register index, and accesses the value in the register.

Notation:

r 8-bit
rr 16-bit

Register File
Register Pair File

Specifies an 8-bit address in the register file, and accesses the 8- or 16-bit value in memory. When specifying a register pair, the address must be even.

Notation:

R 8-bit
RR 16-bit

Editor's Note: Research is needed to determine what happens if a register file pair operand has the lowest bit set. The assembler refuses to process these.

Register Indirect
Register Pair Indirect

Specifies a general-purpose register or register pair using a 3-bit register index, regards the value in the register as an address, and accesses the 8- or 16-bit memory at that address.

Notation:

@r 8-bit
@rr 16-bit

Register Indirect Auto Increment
Register Pair Indirect Auto Increment

Specifies a general-purpose register or register pair using a 3-bit register index, regards the value in the register as an address, accesses the 8- or 16-bit memory at that address, then increases the value in the register by 1.

Notation:

(r)+ 8-bit
(rr)+ 16-bit

Editor's Note: Research is needed to confirm or correct this addressing mode's operation.

Register Indirect Auto Decrement
Register Pair Indirect Auto Decrement

Specifies a general-purpose register or register pair using a 3-bit register index, decreases the value in the register by 1, regards the new value in the register as an address, then accesses the value in memory at that address.

Notation:

-(r) 8-bit
-(rr) 16-bit

Editor's Note: Research is needed to confirm or correct this addressing mode's operation.

Register Index
Register Pair Index
Index Indirect

Specifies a general-purpose register or register pair using a 3-bit register index and a signed 8- or 16-bit displacement offset, regards the sum of the value in the register and the displacement offset as an address, then accesses the value in memory at that address.

Notation:

n(r)	8-bit
nn(rr)	16-bit
@nn(r)	Index Indirect (see below)

If the displacement offset is zero, `n(r)` may be written as `@r` and `nn(rr)` may be written as `@rr`.

The Index Indirect version of this operand type specifies an 8-bit general-purpose register and a 16-bit displacement offset. As this operand type is used only by the CALL and JMP instructions, the leading `@` character indicates that the target code is at the address given by the value loaded from memory.

`r0` and `rr0` cannot be used with this operand type.

Editor's Note: What would be the binary coding for `n(r0)` when used by BBC and BBS is instead interpreted as direct special page. It is not possible for the CPU to use `r0` with this operand type.

Editor's Note: What would be the binary codings for `nn(rr0)` and `@nn(r0)` are instead interpreted as direct indirect. It is not possible for the CPU to use `r0` or `rr0` with this operand type.

Editor's Note: Research is needed to determine why `n(r0)` is prohibited for instructions other than `BBC` and `BBS`. It is rejected by the assembler, but doesn't appear to correspond to any alternate addressing mode.

Specifies a literal numeric value for the operand.

Notation:

#*number*

Immediate specifies an 8-bit number and Immediate Long a 16-bit number. Bit specifies a 3-bit number that represents the position of the bit of interest within its byte of data.

Relative

Specifies an 8-bit signed displacement offset to add to PC to compute the address of the next instruction. The value to use for `PC` in the computation is the address of the byte following the current instruction.

Notation:

RA

The operand is most commonly displayed to the user as the computed or symbolic destination rather than the displacement offset.

Direct
Direct Short
Direct Special Page
Direct Indirect

Specifies a direct address.

Notation:

DA	`0x0000` - `0xFFFF`	Direct
DAs	`0x1000` - `0x1FFF`	Direct Short
DAp	`0xFF00` - `0xFFFF`	Direct Special Page
@DA	`0x0000` - `0xFFFF`	Direct Indirect (see below)

The data at the destination of this operand type depends on the instruction being used.

For Direct Short, the address is formed by taking `0x1000` and adding to it 12 bits that are encoded into the instruction code.

For Direct Special Page, the address is formed by taking `0xFF00` and adding to it 8 bits that are encoded into the instruction code.

The address ultimately represented by the Direct Indirect version of this operand type is the 16-bit value located at the address given by the operand.

Memory and Register

Instructions

Mnemonic	C	Z	S	V	Name	Operation
BCLR					Bit Clear	*dest* = 0
BMOV					Bit Move	*dest* = *src*
BSET				1	Bit Set	*dest* = 1
CLR					Clear	*dest* = 0
CLRC	0				Clear Carry Flag	C = 0
MOV					Move	*dest* = *src*
MOVM					Move Under Mask	*dest* = (*dest* AND *imm*) OR *src*
MOVW					Move Word	*dest* = *src*
POP					Pop from Stack	Remove from stack
POPW					Pop Word from Stack	Remove from stack

PUSH		Push to Stack	Add to stack
PUSHW		Push Word to Stack	Add to stack
SETC	1	Set Carry Flag	`C` = 1
SWAP		Swap Nibbles	*dest* = (*dest* << 4) OR (*dest* >> 4 (zero-filling))

`BCLR` - Bit Clear

Mnemonic	C	Z	S	V	Name	Operation
BCLR					Bit Clear	*dest* = 0

Clears the state of a bit in memory to 0.

Code	Cycles	Operands
00011100 ??000bbb aaaaaaaa		Direct Special Page, Bit
00011100 ??rrrbbb nnnnnnnn		Register Index, Bit [note]
10100bbb RRRRRRRR		Register File, Bit

[note] Cannot be used with `n(r0)`.

`BMOV` - Bit Move

Mnemonic	C	Z	S	V	Name	Operation
BMOV					Bit Move	*dest* = *src*

Moves a bit value into the bit flag (`B`) or memory.

Code	Cycles	Operands
01001110 ?0???bbb RRRRRRRR		Register File, Bit, `BF`
01001110 ?1???bbb RRRRRRRR		`BF`, Register File, Bit

`BSET` - Bit Set

Mnemonic	C	Z	S	V	Name	Operation
BSET					Bit Set	*dest* = 1

Sets the state of a bit in memory to 1.

Code	Cycles	Operands
00011101 ??000bbb aaaaaaaa		Direct Special Page, Bit
00011101 ??rrrbbb nnnnnnnn		Register Index, Bit [note]
10101bbb RRRRRRRR		Register File, Bit

[note] Cannot be used with `n(r0)`.

`CLR` - Clear

Mnemonic	C	Z	S	V	Name	Operation
CLR					Clear	*dest* = 0

Sets a value in memory to 0.

Code	Cycles	Operands
00000000 DDDDDDDD		Register File
00011010 ??ddd000		Register Indirect

`CLRC` - Clear Carry Flag

Mnemonic	C	Z	S	V	Name	Operation
CLRC					Clear Carry Flag	`C` = 0

Clears the carry flag in PS1.

Code	Cycles	Operands
11111010		None

`MOV` - Move

Mnemonic	C	Z	S	V	Name	Operation
MOV					Move	*dest* = *src*

Takes the byte value from *src* and stores it into *dest*.

Code	Cycles	Operands
00101000 00dddsss		Register, Register Indirect
00101000 01dddsss		Register, Register Indirect Auto Increment
00101000 10dddsss nnnnnnnn		Register, Register Index [note 1]
00101000 11dddsss		Register, Register Indirect Auto Decrement
00101001 00sssddd		Register Indirect, Register
00101001 01sssddd		Register Indirect Auto Increment , Register
00101001 10sssddd nnnnnnnn		Register Index, Register [note 1]
00101001 11sssddd		Register Indirect Auto Decrement , Register
00101110 iiiiiiii		`PS0`, Immediate
00111000 00dddsss		Register, Register Pair Indirect
00111000 01dddsss		Register, Register Pair Indirect Auto Increment
00111000 10ddd000 aaaaaaaa aaaaaaaa		Register, Direct Indirect
00111000 10dddsss nnnnnnnn nnnnnnnn		Register, Register Pair Index [note 2]
00111000 11dddsss		Register, Register Pair Indirect Auto Decrement
00111001 00sssddd		Register Pair Indirect, Register

00111001 01sssddd	Register Pair Indirect Auto Increment , Register
00111001 10sss000 aaaaaaaa aaaaaaaa	Direct Indirect, Register
00111001 10sssddd nnnnnnnn nnnnnnnn	Register Pair Index, Register [note 2]
00111001 11sssddd	Register Pair Indirect Auto Decrement , Register
01001000 SSSSSSSS DDDDDDDD	Register File, Register File
01011000 iiiiiiii DDDDDDDD	Register File, Immediate
01011011 00???ddd iiiiiiii	Register Indirect, Immediate
01011011 01???ddd iiiiiiii	Register Indirect Auto Increment , Immediate
01011011 10???ddd nnnnnnnn iiiiiiii	Register Index, Immediate [note 1]
01011011 11???ddd iiiiiiii	Register Indirect Auto Decrement , Immediate
10110ddd SSSSSSSS	Register, Register File
10111sss DDDDDDDD	Register File, Register
11000ddd iiiiiiii	Register, Immediate
11001ddd iiiiiiii	(See below), Immediate

[note 1] Cannot be used with `n(r0)`.
[note 2] Cannot be used with `nn(rr0)`.

The final variant of this instruction specifies a destination with a 3-bit field, as follows:

0 IE0
1 IE1
2 IR0
3 IR1
4 P0
5 P1
6 P2
7 P3

`MOVM` - Move Under Mask

Mnemonic	C	Z	S	V	Name	Operation
MOVM					Move Under Mask	*dest* = (*dest* AND *imm*) OR *src*

Takes the byte value from *dest*, takes the bitwise AND of it with the mask value, takes the bitwise OR of the result and *src*, and stores the result into *dest*.

Code	Cycles	Operands
01011110 DDDDDDDD mmmmmmmm SSSSSSSS		Register File, Immediate, Register File
01011111 DDDDDDDD mmmmmmmm iiiiiiii		Register File, Immediate, Immediate

`MOVW` - Move Word

Mnemonic	C	Z	S	V	Name	Operation
MOVW					Move Word	*dest* = *src*

Takes the word value from *src* and stores it into *dest*.

Code	Cycles	Operands
00111010 00dddsss		Register Pair, Register Pair Indirect
00111010 01dddsss		Register Pair, Register Pair Indirect Auto Increment
00111010 10ddd000 aaaaaaaa aaaaaaaa		Register Pair, Direct Indirect
00111010 10dddsss nnnnnnnn nnnnnnnn		Register Pair, Register Pair Index [note]
00111010 11dddsss		Register Pair, Register Pair Indirect Auto Decrement
00111011 00sssddd		Register Pair Indirect, Register Pair
00111011 01sssddd		Register Pair Indirect Auto Increment , Register Pair
00111011 10sss000 aaaaaaaa aaaaaaaa		Direct Indirect, Register Pair
00111011 10sssddd nnnnnnnn nnnnnnnn		Register Pair Index, Register Pair [note]
00111011 11sssddd		Register Pair Indirect Auto Decrement , Register Pair
00111100 ??dddsss		Register Pair, Register Pair
01001010 SSSSSSS? DDDDDDD?		Register Pair File, Register Pair File
01001011 DDDDDDD? iiiiiiii iiiiiiii		Register Pair File, Immediate Long
01111ddd iiiiiiii iiiiiiii		Register Pair, Immediate Long

[note] Cannot be used with `nn(rr0)`.

`POP` - Pop from Stack

Mnemonic	C	Z	S	V	Name	Operation
POP					Pop from Stack	Remove from stack

Retrieves an 8-bit value from the stack and stores it in the operand. During a pop operation, the value is loaded from the memory pointed to by `SP`, then `SP` is incremented by 1.

Code	Cycles	Operands
00001111 DDDDDDDD		Register File
00011011 ??ddd111		Register

`POPW` - Pop Word from Stack

Mnemonic	C	Z	S	V	Name	Operation
POPW					Pop Word from Stack	Remove from stack

Retrieves a 16-bit value from the stack and stores it in the operand. During a pop operation, the value is loaded from the memory pointed to by SP, then SP is incremented by 2.

Code	Cycles	Operands
00011111 DDDDDDD?		Register Pair File

PUSH - Push to Stack

Mnemonic	C	Z	S	V	Name	Operation
PUSH					Push to Stack	Add to stack

Stores an 8-bit value on the stack. During a push operation, SP is decremented by 1, then the value is stored into memory at the address pointed to by SP.

Code	Cycles	Operands
00001110 SSSSSSSS		Register File
00011011 ??sss110		Register

PUSHW - Push Word to Stack

Mnemonic	C	Z	S	V	Name	Operation
PUSHW					Push Word to Stack	Add to stack

Stores a 16-bit value on the stack. During a push operation, SP is decremented by 2, then the value is stored into memory at the address pointed to by SP.

Code	Cycles	Operands
00011110 SSSSSSS?		Register Pair File

SETC - Set Carry Flag

Mnemonic	C	Z	S	V	Name	Operation
SETC					Set Carry Flag	C = 1

Sets the carry flag (C) to 1.

Code	Cycles	Operands
11111100		None

SWAP - Swap Nibbles

Mnemonic	C	Z	S	V	Name	Operation
SWAP					Swap Nibbles	*dest* = (*dest* << 4) OR (*dest* >> 4 (zero-filling))

Exchanges the nibbles in the operand byte.

Code	Cycles	Operands
00001101 DDDDDDDD		Register File
00011011 ??ddd101		Register Indirect

Arithmetic

Instructions

Mnemonic C Z S V	Name	Operation
ADC	Add with Carry	*dest* = *dest* + *src* + c
ADCW	Add Word with Carry	*dest* = *dest* + *src* + c
ADD	Add	*dest* = *dest* + *src*
ADDW	Add Word	*dest* = *dest* + *src*
BCMP	Bit Compare	(discard) = B - *src*
CMP	Compare	(discard) = *dest* - *src*
CMPW	Compare Word	(discard) = *dest* - *src*
DA	Decimal Adjust	
DEC	Decrement	*dest* = *dest* - 1
DECW	Decrement Word	*dest* = *dest* - 1
DIV	Divide	*src.hi* = *dest* MOD *src.lo*, *dest* = *dest* / *src.lo*
EXTS	Extend Sign	Sign-extend low byte of *dest*
INC	Increment	*dest* = *dest* + 1
INCW	Increment Word	*dest* = *dest* + 1
MULT	Multiply	*dest* = *dest.lo* * *src*
NEG	Negate	*dest* = 0 - *dest*
SBC	Subtract with Carry	*dest* = *dest* - *src* - c
SBCW	Subtract Word with Carry	*dest* = *dest* - *src* - c
SUB	Subtract	*dest* = *dest* - *src*
SUBW	Subtract Word	*dest* = *dest* - *src*

ADC - Add with Carry

Mnemonic C Z S V	Name	Operation
ADC	Add with Carry	*dest* = *dest* + *src* + c

Adds the two operands, plus 1 more if the carry flag (c) was set, and stores the result into *dest*.

Code	Cycles	Operands
00010011 ??dddsss		Register, Register
00100011 00dddsss		Register, Register Indirect
00100011 01dddsss		Register, Register Indirect Auto Increment
00100011 10dddsss nnnnnnnn		Register, Register Index [note 1]

00100011 11dddsss	Register, Register Indirect Auto Decrement
00110011 00dddsss	Register, Register Pair Indirect
00110011 01dddsss	Register, Register Pair Indirect Auto Increment
00110011 10ddd000 aaaaaaaa aaaaaaaa	Register, Direct Indirect
00110011 10dddsss nnnnnnnn nnnnnnnn	Register, Register Pair Index [note 2]
00110011 11dddsss	Register, Register Pair Indirect Auto Decrement
01000011 SSSSSSSS DDDDDDDD	Register File, Register File
01010011 iiiiiiii DDDDDDDD	Register File, Immediate

[note 1] Cannot be used with `n(r0)`.
[note 2] Cannot be used with `nn(rr0)`.

`ADCW` - Add Word with Carry

Mnemonic	C	Z	S	V	Name	Operation
ADCW					Add Word with Carry	*dest* = *dest* + *src* + `c`

Adds the two operands, plus 1 more if the carry flag (`c`) was set, and stores the result into *dest*.

Code	Cycles	Operands
01100011 SSSSSSS? DDDDDDD?		Register Pair File, Register Pair File
01101011 DDDDDDD? iiiiiiii iiiiiiii		Register Pair File, Immediate Long

`ADD` - Add

Mnemonic	C	Z	S	V	Name	Operation
ADD					Add	*dest* = *dest* + *src*

Adds the two operands and stores the result into *dest*.

Code	Cycles	Operands
00010001 ??dddsss		Register, Register
00100001 00dddsss		Register, Register Indirect
00100001 01dddsss		Register, Register Indirect Auto Increment
00100001 10dddsss nnnnnnnn		Register, Register Index [note 1]
00100001 11dddsss		Register, Register Indirect Auto Decrement
00110001 00dddsss		Register, Register Pair Indirect
00110001 01dddsss		Register, Register Pair Indirect Auto Increment
00110001 10ddd000 aaaaaaaa aaaaaaaa		Register, Direct Indirect
00110001 10dddsss nnnnnnnn nnnnnnnn		Register, Register Pair Index [note 2]
00110001 11dddsss		Register, Register Pair Indirect Auto Decrement
01000001 SSSSSSSS DDDDDDDD		Register File, Register File
01010001 iiiiiiii DDDDDDDD		Register File, Immediate

[note 1] Cannot be used with `n(r0)`.
[note 2] Cannot be used with `nn(rr0)`.

`ADDW` - Add Word

Mnemonic	C	Z	S	V	Name	Operation
ADDW					Add Word	*dest* = *dest* + *src*

Adds the two operands and stores the result into *dest*.

Code	Cycles	Operands
01100001 SSSSSSS? DDDDDDD?		Register Pair File, Register Pair File
01101001 DDDDDDD? iiiiiiii iiiiiiii		Register Pair File, Immediate Long

`BCMP` - Bit Compare

Mnemonic	C	Z	S	V	Name	Operation
BCMP					Bit Compare (discard)	= `B` - *src*

Compares the bit flag (`B`) with a bit in memory.

Code	Cycles	Operands
01001111 00???bbb SSSSSSSS		`BF`, Register File, Bit

`CMP` - Compare

Mnemonic	C	Z	S	V	Name	Operation
CMP					Compare (discard)	= *dest* - *src*

Subtracts the operands, updating the status flags in the process, and discards the result.

Code	Cycles	Operands
00010000 ??dddsss		Register, Register
00100000 00dddsss		Register, Register Indirect
00100000 01dddsss		Register, Register Indirect Auto Increment
00100000 10dddsss nnnnnnnn		Register, Register Index [note 1]
00100000 11dddsss		Register, Register Indirect Auto Decrement
00110000 00dddsss		Register, Register Pair Indirect
00110000 01dddsss		Register, Register Pair Indirect Auto Increment
00110000 10ddd000 aaaaaaaa aaaaaaaa		Register, Direct Indirect
00110000 10dddsss nnnnnnnn nnnnnnnn		Register, Register Pair Index [note 2]
00110000 11dddsss		Register, Register Pair Indirect Auto Decrement
01000000 SSSSSSSS DDDDDDDD		Register File, Register File

01010000 iiiiiiii DDDDDDDD	Register File, Immediate
01011010 00???ddd iiiiiiii	Register Indirect, Immediate
01011010 01???ddd iiiiiiii	Register Indirect Auto Increment , Immediate
01011010 10???ddd nnnnnnnn iiiiiiii	Register Index, Immediate [note 1]
01011010 11???ddd iiiiiiii	Register Indirect Auto Decrement , Immediate

[note 1] Cannot be used with `n(r0)`.
[note 2] Cannot be used with `nn(rr0)`.

`CMPW` - Compare Word

Mnemonic	C	Z	S	V	Name	Operation
CMPW					Compare Word (discard)	= *dest* - *src*

Subtracts the operands, updating the status flags in the process, and discards the result.

Code	Cycles	Operands
01100000 SSSSSSS? DDDDDDD?		Register Pair File, Register Pair File
01101000 DDDDDDD? iiiiiiii iiiiiiii		Register Pair File, Immediate Long

`DA` - Decimal Adjust

Mnemonic	C	Z	S	V	Name	Operation
DA					Decimal Adjust	

Editor's Note: Research is needed to determine the exact operation of this instruction. Its apparent use is to correct for BCD after addition or subtraction.

Code	Cycles	Operands
00001100 DDDDDDDD		Register File
00011011 ??ddd100		Register Indirect

`DEC` - Decrement

Mnemonic	C	Z	S	V	Name	Operation
DEC					Decrement	*dest* = *dest* - 1

Subtracts 1 from the operand, storing the result into *dest*.

Code	Cycles	Operands
00001001 DDDDDDDD		Register File
00011011 ??ddd001		Register Indirect

`DECW` - Decrement Word

Mnemonic	C	Z	S	V	Name	Operation
DECW					Decrement Word	*dest* = *dest* - 1

Subtracts 1 from the operand, storing the result into *dest*.

Code	Cycles	Operands
00011001 DDDDDDD?		Register Pair File

DIV - Divide

Mnemonic	C	Z	S	V	Name	Operation
DIV					Divide	*src.hi* = *dest* MOD *src.lo*, *dest* = *dest* / *src.lo*

Performs a division on the 16-bits of *dest* by the low 8 bits of *src*. The 16-bit quotient is stored into *dest* and the 8-bit remainder is stored into the high 8 bits of *src*.

Editor's Note: Research is needed to determine the exact operation of this instruction. One source suggests the following: 1) the division is signed, 2) if the divisor is zero, V is set and no operation or exception occurs, and 3) if *src* is immediate, the remainder is discarded.

Code	Cycles	Operands
01011100 SSSSSSS? DDDDDDD?		Register Pair File, Register Pair File
01011101 iiiiiiii DDDDDDD?		Register Pair File, Immediate

EXTS - Extend Sign

Mnemonic	C	Z	S	V	Name	Operation
EXTS					Extend Sign	Sign-extend low byte of *dest*

If bit 7 of *dest* is clear, the upper 8 bits of *dest* are cleared. Otherwise, the upper 8 bits of *dest* are set. The result is stored into *dest*.

Code	Cycles	Operands
00101100 DDDDDDD?		Register Pair File

INC - Increment

Mnemonic	C	Z	S	V	Name	Operation
INC					Increment	*dest* = *dest* + 1

Adds 1 to the operand, storing the result into *dest*.

Code	Cycles	Operands
00001000 DDDDDDDD		Register File
00011011 ??ddd000		Register Indirect

INCW - Increment Word

Mnemonic	C	Z	S	V	Name	Operation
INCW					Increment Word	*dest* = *dest* + 1

Adds 1 to the operand, storing the result into *dest*.

Code	Cycles	Operands
00011000 DDDDDDD?		Register Pair File

MULT - Multiply

Mnemonic	C	Z	S	V	Name	Operation
MULT					Multiply	*dest* = *dest.lo* * *src*

Multiplies the low 8 bits of *dest* by *src* and stores the 16-bit result into *dest*.

Editor's Note: Research is needed to determine the exact operation of this instruction. One resource suggests that the multiplication is signed.

Code	Cycles	Operands
01001100 SSSSSSSS DDDDDDD?		Register Pair File, Register File
01001101 iiiiiiii DDDDDDD?		Register Pair File, Immediate

NEG - Negate

Mnemonic	C	Z	S	V	Name	Operation
Negate					Negate	*dest* = 0 - *dest*

Calculates the additive inverse of *dest* by subtracting *dest* from zero, and stores the result back into *dest*.

Code	Cycles	Operands
00000001 DDDDDDDD		Register File
00011010 ??ddd001		Register Indirect

SBC - Subtract with Carry

Mnemonic	C	Z	S	V	Name	Operation
SBC					Subtract with Carry	*dest* = *dest* - *src* - c

Subtracts the two operands, minus 1 more if the carry flag (c) was set, and stores the result into *dest*.

Code	Cycles	Operands
00010100 ??dddsss		Register, Register

00100100 00dddsss	Register, Register Indirect
00100100 01dddsss	Register, Register Indirect Auto Increment
00100100 10dddsss nnnnnnnn	Register, Register Index [note 1]
00100100 11dddsss	Register, Register Indirect Auto Decrement
00110100 00dddsss	Register, Register Pair Indirect
00110100 01dddsss	Register, Register Pair Indirect Auto Increment
00110100 10ddd000 aaaaaaaa aaaaaaaa	Register, Direct Indirect
00110100 10dddsss nnnnnnnn nnnnnnnn	Register, Register Pair Index [note 2]
00110100 11dddsss	Register, Register Pair Indirect Auto Decrement
01000100 SSSSSSSS DDDDDDDD	Register File, Register File
01010100 iiiiiiii DDDDDDDD	Register File, Immediate

[note 1] Cannot be used with `n(r0)`.
[note 2] Cannot be used with `nn(rr0)`.

`SBCW` - Subtract Word with Carry

Mnemonic	C	Z	S	V	Name	Operation
SBCW					Subtract Word with Carry	*dest* = *dest* - *src* - `c`

Subtracts the two operands, minus 1 more if the carry flag (`c`) was set, and stores the result into *dest*.

Code	Cycles	Operands
01100100 SSSSSSS? DDDDDDD?		Register Pair File, Register Pair File
01101100 DDDDDDD? iiiiiiii iiiiiiii		Register Pair File, Immediate Long

`SUB` - Subtract

Mnemonic	C	Z	S	V	Name	Operation
SUB					Subtract	*dest* = *dest* - *src*

Subtract the two operands and stores the result into *dest*.

Code	Cycles	Operands
00010010 ??dddsss		Register, Register
00100010 00dddsss		Register, Register Indirect
00100010 01dddsss		Register, Register Indirect Auto Increment
00100010 10dddsss nnnnnnnn		Register, Register Index [note 1]
00100010 11dddsss		Register, Register Indirect Auto Decrement
00110010 00dddsss		Register, Register Pair Indirect
00110010 01dddsss		Register, Register Pair Indirect Auto Increment
00110010 10ddd000 aaaaaaaa aaaaaaaa		Register, Direct Indirect
00110010 10dddsss nnnnnnnn nnnnnnnn		Register, Register Pair Index [note 2]

00110010 11dddsss	Register, Register Pair Indirect Auto Decrement
01000010 SSSSSSSS DDDDDDDD	Register File, Register File
01010010 iiiiiiii DDDDDDDD	Register File, Immediate

[note 1] Cannot be used with `n(r0)`.
[note 2] Cannot be used with `nn(rr0)`.

`SUBW` - Subtract Word

Mnemonic	C	Z	S	V	Name	Operation
SUBW					Subtract Word	*dest* = *dest* - *src*

Subtracts the two operands and stores the result into *dest*.

Code	Cycles	Operands
01100010 SSSSSSS? DDDDDDD?		Register Pair File, Register Pair File
01101010 DDDDDDD? iiiiiiii iiiiiiii		Register Pair File, Immediate Long

Bitwse

Instructions

Mnemonic	C	Z	S	V	Name	Operation
AND					Logical And	*dest* = *dest* AND *src*
ANDW					Logical And Word	*dest* = *dest* AND *src*
BAND					Bit And	`B` = `B` AND *src*
BOR					Bit Or	`B` = `B` OR *src*
BTST					Bit Test	(discard) = `B` AND *src*
BXOR					Bit Exclusive Or	`B` = `B` XOR *src*
COM					Complement	*dest* = NOT *dest*
COMC	✓				Complement Carry Flag	`C` = NOT `C`
OR					Logical Or	*dest* = *dest* OR *src*
ORW					Logical Or Word	*dest* = *dest* OR *src*
RL					Rotate Left	*dest* = (*dest* << 1) OR (*dest* >> 7 (zero-filling))
RLC	✓				Rotate Left through Carry	*dest'* = (*dest* << 1) OR `C`, `C'` = *dest* >> 7 (zero-filling)
RR					Rotate Right	*dest* = (*dest* << 7) OR (*dest* >> 1 (zero-filling))
RRC	✓				Rotate Right through Carry	*dest'* = (`C` << 7) OR (*dest* >> 1 (zero-filling)), `C'` = *dest* AND 1
SLL					Shift Left Logical	*dest* = *dest* << 1
SRA					Shift Right Arithmetic	*dest* = *dest* >> 1 (sign-propagating)
SRL					Shift Right Logical	*dest* = *dest* >> 1 (zero-filling)
XOR					Logical Exclusive Or	*dest* = *dest* XOR *src*

XORW	Logical Exclusive Or Word	*dest* = *dest* XOR *src*

AND - Logical And

Mnemonic	C	Z	S	V	Name	Operation
AND					Logical And	*dest* = *dest* AND *src*

Computes the bitwise AND of the operands. The result is stored in *dest*.

Code	Cycles	Operands
00010101 ??dddsss		Register, Register
00100101 00dddsss		Register, Register Indirect
00100101 01dddsss		Register, Register Indirect Auto Increment
00100101 10dddsss nnnnnnnn		Register, Register Index [note 1]
00100101 11dddsss		Register, Register Indirect Auto Decrement
00110101 00dddsss		Register, Register Pair Indirect
00110101 01dddsss		Register, Register Pair Indirect Auto Increment
00110101 10ddd000 aaaaaaaa aaaaaaaa		Register, Direct Indirect
00110101 10dddsss nnnnnnnn nnnnnnnn		Register, Register Pair Index [note 2]
00110101 11dddsss		Register, Register Pair Indirect Auto Decrement
01000101 SSSSSSSS DDDDDDDD		Register File, Register File
01010101 iiiiiiii DDDDDDDD		Register File, Immediate

[note 1] Cannot be used with `n(r0)`.
[note 2] Cannot be used with `nn(rr0)`.

ANDW - Logical And Word

Mnemonic	C	Z	S	V	Name	Operation
ANDW					Logical And Word	*dest* = *dest* AND *src*

Computes the bitwise AND of the operands. The result is stored in *dest*.

Code	Cycles	Operands
01100101 SSSSSSS? DDDDDDD?		Register Pair File, Register Pair File
01101101 DDDDDDD? iiiiiiii iiiiiiii		Register Pair File, Immediate Long

BAND - Bit And

Mnemonic	C	Z	S	V	Name	Operation
BAND					Bit And	`B` = `B` AND *src*

Computes the bitwise AND of the Bit flag (`B`) and the operand. The result is stored in `B`.

Code	Cycles	Operands
01001111 01???bbb SSSSSSSS		BF, Register File, Bit

BOR - Bit Or

Mnemonic	C	Z	S	V	Name	Operation
BOR					Bit Or	B = B OR *src*

Computes the bitwise OR of the Bit flag (B) and the operand. The result is stored in B.

Code	Cycles	Operands
01001111 10???bbb SSSSSSSS		BF, Register File, Bit

BTST - Bit Test

Mnemonic	C	Z	S	V	Name	Operation
BTST					Bit And (discard)	= *dest* AND *src*

Computes the bitwise AND of the operands, updating the status flags in the process, and discards the result.

Code	Cycles	Operands
00101111 DDDDDDDD iiiiiiii		Register File, Immediate

BXOR - Bit Exclusive Or

Mnemonic	C	Z	S	V	Name	Operation
BXOR					Bit Exclusive Or	B = B XOR *src*

Computes the bitwise exclusive OR of the Bit flag (B) and the operand. The result is stored in B.

Code	Cycles	Operands
01001111 11???bbb SSSSSSSS		BF, Register File, Bit

COM - Complement

Mnemonic	C	Z	S	V	Name	Operation
COM					Complement	*dest* = NOT *dest*

Inverts the bits of *dest* and stores the result back into *dest*.

Code	Cycles	Operands
00000010 DDDDDDDD		Register File
00011010 ??ddd010		Register Indirect

`COMC` - Complement Carry Flag

Mnemonic	C	Z	S	V	Name	Operation
COMC	✓				Complement Carry Flag	`c` = NOT `c`

Inverts the carry flag (`c`).

Code	Cycles	Operands
11111011		None

`OR` - Logical Or

Mnemonic	C	Z	S	V	Name	Operation
OR					Logical Or	*dest* = *dest* OR *src*

Computes the bitwise OR of the operands. The result is stored in *dest*.

Code	Cycles	Operands
00010110 ??dddsss		Register, Register
00100110 00dddsss		Register, Register Indirect
00100110 01dddsss		Register, Register Indirect Auto Increment
00100110 10dddsss nnnnnnnn		Register, Register Index [note 1]
00100110 11dddsss		Register, Register Indirect Auto Decrement
00110110 00dddsss		Register, Register Pair Indirect
00110110 01dddsss		Register, Register Pair Indirect Auto Increment
00110110 10ddd000 aaaaaaaa aaaaaaaa		Register, Direct Indirect
00110110 10dddsss nnnnnnnn nnnnnnnn		Register, Register Pair Index [note 2]
00110110 11dddsss		Register, Register Pair Indirect Auto Decrement
01000110 SSSSSSSS DDDDDDDD		Register File, Register File
01010110 iiiiiiii DDDDDDDD		Register File, Immediate

[note 1] Cannot be used with `n(r0)`.
[note 2] Cannot be used with `nn(rr0)`.

`ORW` - Logical Or Word

Mnemonic	C	Z	S	V	Name	Operation
ORW					Logical Or Word	*dest* = *dest* OR *src*

Computes the bitwise OR of the operands. The result is stored in *dest*.

Code	Cycles	Operands
01100110 SSSSSSS? DDDDDDD?		Register Pair File, Register Pair File
01101110 DDDDDDD? iiiiiiii iiiiiiii		Register Pair File, Immediate Long

RL - Rotate Left

Mnemonic	C	Z	S	V	Name	Operation
RL					Rotate Left	*dest* = (*dest* << 1) OR (dest >> 7 (zero-filling))

Performs a left bitwise rotation on *dest* by 1 bit and stores the result into *dest*.

Code	Cycles	Operands
00000100 DDDDDDDD		Register File
00011010 ??ddd100		Register Indirect

RLC - Rotate Left through Carry

Mnemonic	C	Z	S	V	Name	Operation
RLC					Rotate Left through Carry	*dest'* = (*dest* << 1) OR c, c' = *dest* >> 7 (zero-filling)

Forms a 9-bit value by taking *dest* as the lowest 8 bits and the carry flag (c) as the highest bit, performs a left bitwise rotation on this value by 1 bit, and stores the result into c (high 1 bit) and *dest* (low 8 bits).

Code	Cycles	Operands
00000110 DDDDDDDD		Register File
00011010 ??ddd110		Register Indirect

RR - Rotate Right

Mnemonic	C	Z	S	V	Name	Operation
RR					Rotate Right	*dest* = (*dest* << 7) OR (dest >> 1 (zero-filling))

Performs a right bitwise rotation on *dest* by 1 bit and stores the result into *dest*.

Code	Cycles	Operands
00000011 DDDDDDDD		Register File
00011010 ??ddd011		Register Indirect

RRC - Rotate Right through Carry

Mnemonic	C	Z	S	V	Name	Operation
RRC					Rotate Right through Carry	*dest'* = (c << 7) OR (*dest* >> 1 (zero-filling)), c' = *dest* AND 1

Forms a 9-bit value by taking *dest* as the lowest 8 bits and the carry flag (c) as the highest bit, performs a right bitwise rotation on this value by 1 bit, and stores the result into c (high 1 bit) and *dest* (low 8 bits).

Code	Cycles	Operands
00000101 DDDDDDDD		Register File
00011010 ??ddd101		Register Indirect

SLL - Shift Left Logical

Mnemonic	C	Z	S	V	Name	Operation
SLL					Shift Left Logical	*dest* = *dest* << 1

Performs a left bitwise shift on *dest* by 1 bit and stores the result in *dest*.

Code	Cycles	Operands
00001011 DDDDDDDD		Register File
00011011 ??ddd011		Register Indirect

SRA - Shift Right Arithmetic

Mnemonic	C	Z	S	V	Name	Operation
SRA					Shift Right Arithmetic	*dest* = *dest* >> 1 (sign-propagating)

Performs a sign-propagating right bitwise shift on *dest* by 1 bit and stores the result in *dest*. The state of the highest-order bit will not change.

Code	Cycles	Operands
00001010 DDDDDDDD		Register File
00011011 ??ddd010		Register Indirect

SRL - Shift Right Logical

Mnemonic	C	Z	S	V	Name	Operation
SRL					Shift Right Logical	*dest* = *dest* >> 1 (zero-filling)

Performs a zero-filling right bitwise shift on *dest* by 1 bit and stores the result in *dest*. The highest-order bit will be cleared.

Code	Cycles	Operands
00000111 DDDDDDDD		Register File
00011010 ??ddd111		Register Indirect

XOR - Logical Exclusive Or

Mnemonic	C	Z	S	V	Name	Operation
XOR					Logical Exclusive Or	*dest* = *dest* XOR *src*

Computes the bitwise exclusive OR of the operands. The result is stored in *dest*.

Code	Cycles	Operands
00010111 ??dddsss		Register, Register
00100111 00dddsss		Register, Register Indirect
00100111 01dddsss		Register, Register Indirect Auto Increment
00100111 10dddsss nnnnnnnn		Register, Register Index [note 1]
00100111 11dddsss		Register, Register Indirect Auto Decrement
00110111 00dddsss		Register, Register Pair Indirect
00110111 01dddsss		Register, Register Pair Indirect Auto Increment
00110111 10ddd000 aaaaaaaa aaaaaaaa		Register, Direct Indirect
00110111 10dddsss nnnnnnnn nnnnnnnn		Register, Register Pair Index [note 2]
00110111 11dddsss		Register, Register Pair Indirect Auto Decrement
01000111 SSSSSSSS DDDDDDDD		Register File, Register File
01010111 iiiiiiii DDDDDDDD		Register File, Immediate

[note 1] Cannot be used with `n(r0)`.
[note 2] Cannot be used with `nn(rr0)`.

`XORW` - Logical Exclusive Or Word

Mnemonic	C	Z	S	V	Name	Operation
XORW					Logical Exclusive Or Word	*dest* = *dest* XOR *src*

Computes the bitwise exclusive OR of the operands. The result is stored in *dest*.

Code	Cycles	Operands
01100111 SSSSSSS? DDDDDDD?		Register Pair File, Register Pair File
01101111 DDDDDDD? iiiiiiii iiiiiiii		Register Pair File, Immediate Long

Jump and Control

Instructions

Mnemonic	C	Z	S	V	Name	Operation
BBC					Branch on Bit Clear	if ((*src* >> *bit*) AND 1) = 0 then jump
BBS					Branch on Bit Set	if ((*src* >> *bit*) AND 1) = 1 then jump
BR					Branch	if condition then jump
CALL					Call Subroutine	Push `PC`, `PC` = *dest*
CALS					Short Call Subroutine	Push `PC`, `PC` = *dest*
DBNZ					Decrement and Branch on Non-Zero	*src* = *src* - 1, if *src* ≠ 0 then jump
DI					Disable Interrupt	`I` = 0
DM						
EI					Enable Interrupt	`I` = 1
HALT					Move to HALT Mode	Wait for interrupt

IRET	* * * *	Return from Interrupt	Pop PS1, PC
JMP		Jump	if condition then jump
NOP		No Operation	None
RET		Return from Subroutine	Pop PC
STOP		Go to STOP Mode	Suspend system and wait for interrupt

BBC - Branch on Bit Clear

Mnemonic	C	Z	S	V	Name	Operation
BBC					Branch on Bit Clear	if ((*src* >> *bit*) AND 1) = 0 then jump

Tests a bit in memory. If the bit is clear, a jump is performed by adding a signed displacement offset to PC.

Code	Cycles	Operands
00101010 ??000bbb aaaaaaaa 00000000		Direct Special Page, Bit, Relative
00101010 ??sssbbb nnnnnnnn 00000000		Register Index, Bit, Relative [note]
10000bbb SSSSSSSS 00000000		Register File, Bit, Relative

[note] Cannot be used with `n(r0)`.

BBS - Branch on Bit Set

Mnemonic	C	Z	S	V	Name	Operation
BBS					Branch on Bit Set	if ((*src* >> *bit*) AND 1) = 1 then jump

Tests a bit in memory. If the bit is set, a jump is performed by adding a signed displacement offset to PC.

Code	Cycles	Operands
00101011 ??000bbb aaaaaaaa 00000000		Direct Special Page, Bit, Relative
00101011 ??sssbbb nnnnnnnn 00000000		Register Index, Bit, Relative [note]
10001bbb SSSSSSSS 00000000		Register File, Bit, Relative

[note] Cannot be used with `n(r0)`.

BR - Branch

Mnemonic	C	Z	S	V	Name	Operation
BR					Branch	if condition then jump

Tests a combination of flags in PS1. If the condition is true, a jump is performed by adding a signed displacement offset to PC.

Code	Cycles	Operands

1101cccc 00000000	Condition, Relative

If `T` is specified as the condition, the condition operand does not need to be written.

`CALL` - Call Subroutine

Mnemonic	C	Z	S	V	Name	Operation
CALL					Call Subroutine	Push `PC`, `PC` = *dest*

Calls a subroutine by saving the return address to the stack and updating `PC` to the address of the subroutine.

Code	Cycles	Operands
00111111 ?0???sss		Register Pair Indirect
00111111 ?1000??? aaaaaaaa aaaaaaaa		Direct Indirect
00111111 ?1sss??? nnnnnnnn nnnnnnnn		Index Indirect [note]
01001001 aaaaaaaa aaaaaaaa		Direct

[note] Cannot be used with `nn(r0)`.

`CALS` - Call Short Subroutine

Mnemonic	C	Z	S	V	Name	Operation
CALS					Call Short Subroutine	Push `PC`, `PC` = *dest*

Calls a subroutine by saving the return address to the stack and updating `PC` to the address of the subroutine.

Code	Cycles	Operands
1110aaaa aaaaaaaa		Direct Short [note]

[note] The highest 4 bits of the operand value comes from the first instruction byte, with the second instruction byte serving as the lower 8 bits.

`DBNZ` - Decrement and Branch on Non-Zero

Mnemonic	C	Z	S	V	Name	Operation
DBNZ					Decrement and Branch on Non-Zero	*src* = *src* - 1, if *src* ≠ 0 then jump

Subtracts 1 from *src* and stores the result into *src*. If the new value of *src* is not zero, a jump is performed by adding a signed displacement offset to `PC`.

Code	Cycles	Operands
01110sss 00000000		Register, Relative

`DI` - Disable Interrupt

Mnemonic	C	Z	S	V	Name	Operation
DI					Disable Interrupt	I = 0

Clears the Interrupt Enable flag (I), thereby masking interrupts.

Code	Cycles	Operands
11111110		None

DM - Unknown

Mnemonic	C	Z	S	V	Name	Operation
DM						

Editor's Note: The operation of this instruction is unknown. The assembler does not produce its opcodes for any inputs, and its existence is known only from a disassembler that was part of the SDK. Research is needed to determine what this instruction does.

Code	Cycles	Operands
00101101 iiiiiiii		Immediate
00111101 RRRRRRRR		Register File

EI - Enable Interrupt

Mnemonic	C	Z	S	V	Name	Operation
EI					Enable Interrupt	I = 1

Sets the Interrupt Enable flag (I), thereby permitting interrupts.

Code	Cycles	Operands
11111101		None

HALT - Move to HALT Mode

Mnemonic	C	Z	S	V	Name	Operation
HALT					Move to HALT Mode	Wait for interrupt

Suspends the CPU until an interrupt is requested. After an interrupt is requested (which may result in a handler routine ending with IRET), execution will continue at the instruction following the HALT instruction.

Code	Cycles	Operands
11110001		None

IRET - Return from Interrupt

Mnemonic	C	Z	S	V	Name	Operation

IRET	*	*	*	*	Return from Interrupt Pop `PS1`, `PC`

Returns from an interrupt handler. The program will resume from wherever it left off: either the instruction following the instruction that raised the exception, or the instruction that would have been executed had an interrupt not occurred.

Code	Cycles	Operands
11111001		None

`JMP` - Jump

Mnemonic	C	Z	S	V	Name	Operation
JMP					Jump	if condition then jump

Stores *dest* into `PC`, optionally after testing a combination of flags in PS1. If a condition is tested and is false, `PC` will not be changed.

Code	Cycles	Operands
00111110 ?0???sss		Register Pair Indirect
00111110 ?1000??? aaaaaaaa aaaaaaaa		Direct Indirect
00111110 ?1sss??? nnnnnnnn nnnnnnnn		Index Indirect [note]
1001cccc aaaaaaaa aaaaaaaa		Condition, Direct

[note] Cannot be used with `nn(r0)`.

If `T` is specified as the condition, the condition operand does not need to be written.

`NOP` - No Operation

Mnemonic	C	Z	S	V	Name	Operation
NOP					No Operation	None

No action is performed.

Code	Cycles	Operands
11111111		None

`RET` - Return from Subroutine

Mnemonic	C	Z	S	V	Name	Operation
RET					Return from Subroutine	Pop `PC`

Returns from a subroutine, restoring `PC` to the location when the subroutine was first called.

Code	Cycles	Operands
11111000		None

STOP - Go to STOP Mode

Mnemonic	C	Z	S	V	Name	Operation
STOP					Go to STOP Mode	Suspend system and wait for interrupt

Suspends most components of the system until an interrupt is requested. After an interrupt is requested (which may result in a handler routine ending with IRET), execution will continue at the instruction following the STOP instruction.

Editor's Note: Research is needed to determine what components are suspended by this instruction.

Code	Cycles	Operands
11110000		None

Opcode Map

Code	Mnemonic	Operands
00000000 DDDDDDDD	CLR	Register File
00000001 DDDDDDDD	NEG	Register File
00000010 DDDDDDDD	COM	Register File
00000011 DDDDDDDD	RR	Register File
00000100 DDDDDDDD	RL	Register File
00000101 DDDDDDDD	RRC	Register File
00000110 DDDDDDDD	RLC	Register File
00000111 DDDDDDDD	SRL	Register File
00001000 DDDDDDDD	INC	Register File
00001001 DDDDDDDD	DEC	Register File
00001010 DDDDDDDD	SRA	Register File
00001011 DDDDDDDD	SLL	Register File
00001100 DDDDDDDD	DA	Register File
00001101 DDDDDDDD	SWAP	Register File
00001110 SSSSSSSS	PUSH	Register File
00001111 DDDDDDDD	POP	Register File
00010000 ??dddsss	CMP	Register, Register
00010001 ??dddsss	ADD	Register, Register
00010010 ??dddsss	SUB	Register, Register
00010011 ??dddsss	ADC	Register, Register
00010100 ??dddsss	SBC	Register, Register
00010101 ??dddsss	AND	Register, Register
00010110 ??dddsss	OR	Register, Register
00010111 ??dddsss	XOR	Register, Register
00011000 DDDDDDD?	INCW	Register Pair File
00011001 DDDDDDD?	DECW	Register Pair File
00011010 ??ddd000	CLR	Register Indirect

00011010 ??ddd001	NEG	Register Indirect
00011010 ??ddd010	COM	Register Indirect
00011010 ??ddd011	RR	Register Indirect
00011010 ??ddd100	RL	Register Indirect
00011010 ??ddd101	RRC	Register Indirect
00011010 ??ddd110	RLC	Register Indirect
00011010 ??ddd111	SRL	Register Indirect
00011011 ??ddd000	INC	Register Indirect
00011011 ??ddd001	DEC	Register Indirect
00011011 ??ddd010	SRA	Register Indirect
00011011 ??ddd011	SLL	Register Indirect
00011011 ??ddd100	DA	Register Indirect
00011011 ??ddd101	SWAP	Register Indirect
00011011 ??sss110	PUSH	Register
00011011 ??ddd111	POP	Register
00011100 ??000bbb aaaaaaaa	BCLR	Direct Special Page, Bit
00011100 ??rrrbbb nnnnnnnn	BCLR	Register Index, Bit
00011101 ??000bbb aaaaaaaa	BSET	Direct Special Page, Bit
00011101 ??rrrbbb nnnnnnnn	BSET	Register Index, Bit
00011110 SSSSSSS?	PUSHW	Register Pair File
00011111 DDDDDDD?	POPW	Register Pair File
00100000 00dddsss	CMP	Register, Register Indirect
00100000 01dddsss	CMP	Register, Register Indirect Auto Increment
00100000 10dddsss nnnnnnnn	CMP	Register, Register Index
00100000 11dddsss	CMP	Register, Register Indirect Auto Decrement
00100001 00dddsss	ADD	Register, Register Indirect
00100001 01dddsss	ADD	Register, Register Indirect Auto Increment
00100001 10dddsss nnnnnnnn	ADD	Register, Register Index
00100001 11dddsss	ADD	Register, Register Indirect Auto Decrement
00100010 00dddsss	SUB	Register, Register Indirect
00100010 01dddsss	SUB	Register, Register Indirect Auto Increment
00100010 10dddsss nnnnnnnn	SUB	Register, Register Index
00100010 11dddsss	SUB	Register, Register Indirect Auto Decrement
00100011 00dddsss	ADC	Register, Register Indirect
00100011 01dddsss	ADC	Register, Register Indirect Auto Increment
00100011 10dddsss nnnnnnnn	ADC	Register, Register Index
00100011 11dddsss	ADC	Register, Register Indirect Auto Decrement
00100100 00dddsss	SBC	Register, Register Indirect
00100100 01dddsss	SBC	Register, Register Indirect Auto Increment
00100100 10dddsss nnnnnnnn	SBC	Register, Register Index

00100100 11dddsss	SBC	Register, Register Indirect Auto Decrement
00100101 00dddsss	AND	Register, Register Indirect
00100101 01dddsss	AND	Register, Register Indirect Auto Increment
00100101 10dddsss nnnnnnnn	AND	Register, Register Index
00100101 11dddsss	AND	Register, Register Indirect Auto Decrement
00100110 00dddsss	OR	Register, Register Indirect
00100110 01dddsss	OR	Register, Register Indirect Auto Increment
00100110 10dddsss nnnnnnnn	OR	Register, Register Index
00100110 11dddsss	OR	Register, Register Indirect Auto Decrement
00100111 00dddsss	XOR	Register, Register Indirect
00100111 01dddsss	XOR	Register, Register Indirect Auto Increment
00100111 10dddsss nnnnnnnn	XOR	Register, Register Index
00100111 11dddsss	XOR	Register, Register Indirect Auto Decrement
00101000 00dddsss	MOV	Register, Register Indirect
00101000 01dddsss	MOV	Register, Register Indirect Auto Increment
00101000 10dddsss nnnnnnnn	MOV	Register, Register Index
00101000 11dddsss	MOV	Register, Register Indirect Auto Decrement
00101001 00sssddd	MOV	Register Indirect, Register
00101001 01sssddd	MOV	Register Indirect Auto Increment, Register
00101001 10sssddd nnnnnnnn	MOV	Register Index, Register
00101001 11sssddd	MOV	Register Indirect Auto Decrement, Register
00101010 ??000bbb aaaaaaaa 00000000	BBC	Direct Special Page, Bit, Relative
00101010 ??sssbbb nnnnnnnn 00000000	BBC	Register Index, Bit, Relative
00101011 ??000bbb aaaaaaaa 00000000	BBS	Direct Special Page, Bit, Relative
00101011 ??sssbbb nnnnnnnn 00000000	BBS	Register Index, Bit, Relative
00101100 DDDDDDD?	EXTS	Register Pair File
00101101 iiiiiiii	DM	Immediate
00101110 iiiiiiii	MOV	PS0, Immediate
00101111 DDDDDDDD iiiiiiii	BTST	Register File, Immediate
00110000 00dddsss	CMP	Register, Register Pair Indirect
00110000 01dddsss	CMP	Register, Register Pair Indirect Auto Increment
00110000 10ddd000 aaaaaaaa aaaaaaaa	CMP	Register, Direct Indirect
00110000 10dddsss nnnnnnnn nnnnnnnn	CMP	Register, Register Pair Index
00110000 11dddsss	CMP	Register, Register Pair Indirect Auto Decrement
00110001 00dddsss	ADD	Register, Register Pair Indirect

00110001 01dddsss	ADD	Register, Register Pair Indirect Auto Increment
00110001 10ddd000 aaaaaaaa aaaaaaaa	ADD	Register, Direct Indirect
00110001 10dddsss nnnnnnnn nnnnnnnn	ADD	Register, Register Pair Index
00110001 11dddsss	ADD	Register, Register Pair Indirect Auto Decrement
00110010 00dddsss	SUB	Register, Register Pair Indirect
00110010 01dddsss	SUB	Register, Register Pair Indirect Auto Increment
00110010 10ddd000 aaaaaaaa aaaaaaaa	SUB	Register, Direct Indirect
00110010 10dddsss nnnnnnnn nnnnnnnn	SUB	Register, Register Pair Index
00110010 11dddsss	SUB	Register, Register Pair Indirect Auto Decrement
00110011 00dddsss	ADC	Register, Register Pair Indirect
00110011 01dddsss	ADC	Register, Register Pair Indirect Auto Increment
00110011 10ddd000 aaaaaaaa aaaaaaaa	ADC	Register, Direct Indirect
00110011 10dddsss nnnnnnnn nnnnnnnn	ADC	Register, Register Pair Index
00110011 11dddsss	ADC	Register, Register Pair Indirect Auto Decrement
00110100 00dddsss	SBC	Register, Register Pair Indirect
00110100 01dddsss	SBC	Register, Register Pair Indirect Auto Increment
00110100 10ddd000 aaaaaaaa aaaaaaaa	SBC	Register, Direct Indirect
00110100 10dddsss nnnnnnnn nnnnnnnn	SBC	Register, Register Pair Index
00110100 11dddsss	SBC	Register, Register Pair Indirect Auto Decrement
00110101 00dddsss	AND	Register, Register Pair Indirect
00110101 01dddsss	AND	Register, Register Pair Indirect Auto Increment
00110101 10ddd000 aaaaaaaa aaaaaaaa	AND	Register, Direct Indirect
00110101 10dddsss nnnnnnnn nnnnnnnn	AND	Register, Register Pair Index
00110101 11dddsss	AND	Register, Register Pair Indirect Auto Decrement
00110110 00dddsss	OR	Register, Register Pair Indirect
00110110 01dddsss	OR	Register, Register Pair Indirect Auto Increment
00110110 10ddd000 aaaaaaaa aaaaaaaa	OR	Register, Direct Indirect
00110110 10dddsss nnnnnnnn nnnnnnnn	OR	Register, Register Pair Index
00110110 11dddsss	OR	Register, Register Pair Indirect Auto Decrement
00110111 00dddsss	XOR	Register, Register Pair Indirect
00110111 01dddsss	XOR	Register, Register Pair Indirect Auto Increment
00110111 10ddd000 aaaaaaaa aaaaaaaa	XOR	Register, Direct Indirect
00110111 10dddsss nnnnnnnn nnnnnnnn	XOR	Register, Register Pair Index

00110111 11dddsss	XOR	Register, Register Pair Indirect Auto Decrement
00111000 00dddsss	MOV	Register, Register Pair Indirect
00111000 01dddsss	MOV	Register, Register Pair Indirect Auto Increment
00111000 10ddd000 aaaaaaaa aaaaaaaa	MOV	Register, Direct Indirect
00111000 10dddsss nnnnnnnn nnnnnnnn	MOV	Register, Register Pair Index
00111000 11dddsss	MOV	Register, Register Pair Indirect Auto Decrement
00111001 00sssddd	MOV	Register Pair Indirect, Register
00111001 01sssddd	MOV	Register Pair Indirect Auto Increment, Register
00111001 10sss000 aaaaaaaa aaaaaaaa	MOV	Direct Indirect, Register
00111001 10sssddd nnnnnnnn nnnnnnnn	MOV	Register Pair Index, Register
00111001 11sssddd	MOV	Register Pair Indirect Auto Decrement, Register
00111010 00dddsss	MOVW	Register Pair, Register Pair Indirect
00111010 01dddsss	MOVW	Register Pair, Register Pair Indirect Auto Increment
00111010 10ddd000 aaaaaaaa aaaaaaaa	MOVW	Register Pair, Direct Indirect
00111010 10dddsss nnnnnnnn nnnnnnnn	MOVW	Register Pair, Register Pair Index
00111010 11dddsss	MOVW	Register Pair, Register Pair Indirect Auto Decrement
00111011 00sssddd	MOVW	Register Pair Indirect, Register Pair
00111011 01sssddd	MOVW	Register Pair Indirect Auto Increment, Register Pair
00111011 10sss000 aaaaaaaa aaaaaaaa	MOVW	Direct Indirect, Register Pair
00111011 10sssddd nnnnnnnn nnnnnnnn	MOVW	Register Pair Index, Register Pair
00111011 11sssddd	MOVW	Register Pair Indirect Auto Decrement, Register Pair
00111100 ??dddsss	MOVW	Register Pair, Register Pair
00111101 RRRRRRRR	DM	Register File
00111110 ?0???sss	JMP	Register Pair Indirect
00111110 ?1000??? aaaaaaaa aaaaaaaa	JMP	Direct Indirect
00111110 ?1sss??? nnnnnnnn nnnnnnnn	JMP	Index Indirect
00111111 ?0???sss	CALL	Register Pair Indirect
00111111 ?1000??? aaaaaaaa aaaaaaaa	CALL	Direct Indirect
00111111 ?1sss??? nnnnnnnn nnnnnnnn	CALL	Index Indirect
01000000 SSSSSSSS DDDDDDDD	CMP	Register File, Register File
01000001 SSSSSSSS DDDDDDDD	ADD	Register File, Register File
01000010 SSSSSSSS DDDDDDDD	SUB	Register File, Register File
01000011 SSSSSSSS DDDDDDDD	ADC	Register File, Register File
01000100 SSSSSSSS DDDDDDDD	SBC	Register File, Register File
01000101 SSSSSSSS DDDDDDDD	AND	Register File, Register File
01000110 SSSSSSSS DDDDDDDD	OR	Register File, Register File

01000111 SSSSSSSS DDDDDDDD	XOR	Register File, Register File
01001000 SSSSSSSS DDDDDDDD	MOV	Register File, Register File
01001001 aaaaaaaa aaaaaaaa	CALL	Direct
01001010 SSSSSSS? DDDDDDD?	MOVW	Register Pair File, Register Pair File
01001011 DDDDDDD? iiiiiiii iiiiiiii	MOVW	Register Pair File, Immediate Long
01001100 SSSSSSSS DDDDDDD?	MULT	Register Pair File, Register File
01001101 iiiiiiii DDDDDDD?	MULT	Register Pair File, Immediate
01001110 ?0???bbb RRRRRRRR	BMOV	Register File, Bit, BF
01001110 ?1???bbb RRRRRRRR	BMOV	BF, Register File, Bit
01001111 00???bbb SSSSSSSS	BCMP	BF, Register File, Bit
01001111 01???bbb SSSSSSSS	BAND	BF, Register File, Bit
01001111 10???bbb SSSSSSSS	BOR	BF, Register File, Bit
01001111 11???bbb SSSSSSSS	BXOR	BF, Register File, Bit
01010000 iiiiiiii DDDDDDDD	CMP	Register File, Immediate
01010001 iiiiiiii DDDDDDDD	ADD	Register File, Immediate
01010010 iiiiiiii DDDDDDDD	SUB	Register File, Immediate
01010011 iiiiiiii DDDDDDDD	ADC	Register File, Immediate
01010100 iiiiiiii DDDDDDDD	SBC	Register File, Immediate
01010101 iiiiiiii DDDDDDDD	AND	Register File, Immediate
01010110 iiiiiiii DDDDDDDD	OR	Register File, Immediate
01010111 iiiiiiii DDDDDDDD	XOR	Register File, Immediate
01011000 iiiiiiii DDDDDDDD	MOV	Register File, Immediate
01011001	Illegal	
01011010 00???ddd iiiiiiii	CMP	Register Indirect, Immediate
01011010 01???ddd iiiiiiii	CMP	Register Indirect Auto Increment, Immediate
01011010 10???ddd nnnnnnnn iiiiiiii	CMP	Register Index, Immediate
01011010 11???ddd iiiiiiii	CMP	Register Indirect Auto Decrement, Immediate
01011011 00???ddd iiiiiiii	MOV	Register Indirect, Immediate
01011011 01???ddd iiiiiiii	MOV	Register Indirect Auto Increment, Immediate
01011011 10???ddd nnnnnnnn iiiiiiii	MOV	Register Index, Immediate
01011011 11???ddd iiiiiiii	MOV	Register Indirect Auto Decrement, Immediate
01011100 SSSSSSS? DDDDDDD?	DIV	Register Pair File, Register Pair File
01011101 iiiiiiii DDDDDDD?	DIV	Register Pair File, Immediate
01011110 DDDDDDDD mmmmmmmm SSSSSSSS	MOVM	Register File, Immediate, Register File
01011111 DDDDDDDD mmmmmmmm iiiiiiii	MOVM	Register File, Immediate, Immediate
01100000 SSSSSSS? DDDDDDD?	CMPW	Register Pair File, Register Pair File
01100001 SSSSSSS? DDDDDDD?	ADDW	Register Pair File, Register Pair File
01100010 SSSSSSS? DDDDDDD?	SUBW	Register Pair File, Register Pair File

01100011 SSSSSSS? DDDDDDD?	ADCW	Register Pair File, Register Pair File
01100100 SSSSSSS? DDDDDDD?	SBCW	Register Pair File, Register Pair File
01100101 SSSSSSS? DDDDDDD?	ANDW	Register Pair File, Register Pair File
01100110 SSSSSSS? DDDDDDD?	ORW	Register Pair File, Register Pair File
01100111 SSSSSSS? DDDDDDD?	XORW	Register Pair File, Register Pair File
01101000 DDDDDDD? iiiiiiii iiiiiiii	CMPW	Register Pair File, Immediate Long
01101001 DDDDDDD? iiiiiiii iiiiiiii	ADDW	Register Pair File, Immediate Long
01101010 DDDDDDD? iiiiiiii iiiiiiii	SUBW	Register Pair File, Immediate Long
01101011 DDDDDDD? iiiiiiii iiiiiiii	ADCW	Register Pair File, Immediate Long
01101100 DDDDDDD? iiiiiiii iiiiiiii	SBCW	Register Pair File, Immediate Long
01101101 DDDDDDD? iiiiiiii iiiiiiii	ANDW	Register Pair File, Immediate Long
01101110 DDDDDDD? iiiiiiii iiiiiiii	ORW	Register Pair File, Immediate Long
01101111 DDDDDDD? iiiiiiii iiiiiiii	XORW	Register Pair File, Immediate Long
01110sss 00000000	DBNZ	Register, Relative
01111ddd iiiiiiii iiiiiiii	MOVW	Register Pair, Immediate Long
10000bbb SSSSSSSS 00000000	BBC	Register File, Bit, Relative
10001bbb SSSSSSSS 00000000	BBS	Register File, Bit, Relative
1001cccc aaaaaaaa aaaaaaaa	JMP	Condition, Direct
10100bbb RRRRRRRR	BCLR	Register File, Bit
10101bbb RRRRRRRR	BSET	Register File, Bit
10110ddd SSSSSSSS	MOV	Register, Register File
10111sss DDDDDDDD	MOV	Register File, Register
11000ddd iiiiiiii	MOV	Register, Immediate
11001ddd iiiiiiii	MOV	(Special register/port), Immediate
1101cccc 00000000	BR	Condition, Relative
1110aaaa aaaaaaaa	CALS	Direct Short
11110000	STOP	
11110001	HALT	
11110010	Illegal	
11110011	Illegal	
11110100	Illegal	
11110101	Illegal	
11110110	Illegal	
11110111	Illegal	
11111000	RET	
11111001	IRET	
11111010	CLRC	
11111011	COMC	
11111100	SETC	
11111101	EI	
11111110	DI	
11111111	NOP	

System Reset

System components are initialized in the following ways when the Game.com is powered on.

CPU

The following registers are initialized on reset:

PC	0x1020
PS1	I = 0, all others undefined
SYS	0x00

All other CPU registers have undefined contents on reset.

Memory Map

The following registers are initialized on reset:

MMU0	0x00
MMU1	0x00
MMU2	0x00
MMU3	0x00
MMU4	0x00

Additionally, a 4 KiB internal boot ROM is mapped to addresses `0x1000` - `0x1FFF` after reset.

About

Game.com - Sacred Tech Scroll
March 10, 2023
Written by Guy Perfect

Special thanks to the following individuals:

Mr^Burns

The Beesh-Spweesh!

zwenergy

References

SM8521
Written by Sharp Corporation
Download

Sharp Micro Controller SM85 Series Macro Assembler
Version 1.10
Created by Sharp Corporation

Dumping Tiger Game.com ROMs
April 1, 2019
Article detailing the dumping process
Written by Kao
Link

March 10, 2023